Lab 34 — File Integrity Monitoring with AIDE

> **SY0-701 Objective 4.4** — Explain security alerting and monitoring concepts and tools.
>
> **Domain 4** — Security Operations (28%) · **Time:** ~20 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Baseline critical files with AIDE, detect a web defacement, a dropped backdoor and a new account, respond by restoring and quarantining, re-baseline after the approved change, and forward the daily check into syslog for the SIEM.

You'll build: A scoped AIDE configuration and a cryptographic baseline database, a tamper report, a quarantine folder holding the backdoor, and a scheduled check whose results reach the system journal.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 05 (hashing) is helpful background; tools installed in step 1.

Background

- **File integrity monitoring (FIM)** is a detective control for the **integrity** leg of the CIA triad. It records hashes and attributes of files and reports any later change.
- AIDE stores, for each watched file, its permissions, inode, owner, group, size, modification and change times and a SHA-256 hash. A trojaned binary changes its hash even if the attacker resets its timestamp.
- The baseline must be taken on a **known-clean** system and stored **read-only or off-host**. An attacker who can rewrite the database can hide anything.
- After an **approved change** (patching, a deployment) you **re-baseline**, otherwise the noise buries the next real alert.
- FIM results should feed the SIEM through syslog or an agent. PCI DSS, for example, requires change detection on critical files.

Steps

1. Install AIDE

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y aide
aide --version 2>&1 | head -1
```

2. Create the content to protect

Simulate a small web application and an admin script. Keep a known-good copy of the web page so you can restore it later.

```
mkdir -p /srv/webapp /root/known-good /root/quarantine
echo '<h1>SecPlus Bank - Welcome</h1>' > /srv/webapp/index.html
printf '[db]\nhost=10.0.20.5\nuser=webapp\n' > /srv/webapp/app.ini
```

```
printf '#!/bin/sh\ntar -czf /tmp/webapp-backup.tgz /srv/webapp\n' > /usr/local/bin/backup-
webapp.sh
chmod 755 /usr/local/bin/backup-webapp.sh
cp -a /srv/webapp/index.html /root/known-good/
```

3. Write a scoped AIDE configuration

A dedicated, small configuration keeps the scan fast and the report readable. `LAB` checks permissions, inode, links, owner, group, size, mtime, ctime and SHA-256.

```
mkdir -p /var/lib/aide/lab
cat > /etc/aide/aide-lab.conf <<'EOF'
database_in=file:/var/lib/aide/lab/aide.db
database_out=file:/var/lib/aide/lab/aide.db.new
gzip_dbout=no
report_url=stdout
LAB = p+i+n+u+g+s+m+c+sha256
/srv/webapp LAB
/usr/local/bin LAB
/etc/passwd LAB
/etc/shadow LAB
/etc/group LAB
/etc/sudoers LAB
EOF
aide --config=/etc/aide/aide-lab.conf --config-check && echo "configuration OK"
```

4. Initialise the baseline and protect it

Create the database on the clean system, promote it to the reference copy, make it read-only and record its own hash "off-host" so that tampering with the baseline is detectable too.

```
aide --config=/etc/aide/aide-lab.conf --init
cp /var/lib/aide/lab/aide.db.new /var/lib/aide/lab/aide.db
chmod 400 /var/lib/aide/lab/aide.db
mkdir -p /root/offhost && sha256sum /var/lib/aide/lab/aide.db | tee /root/offhost/aide-
db.sha256
aide --config=/etc/aide/aide-lab.conf --check; echo "clean check exit code: $?"
```

A clean check returns exit code `0`.

5. Simulate an intrusion

The attacker defaces the web page, drops a SUID backdoor next to the admin script and creates a persistence account.

```
echo '<h1>HACKED by lab-actor</h1>' > /srv/webapp/index.html
cp /bin/sh /usr/local/bin/.sysupdate && chmod 4755 /usr/local/bin/.sysupdate
useradd -m -s /bin/bash svc-update
```

6. Detect the changes

Run the check. AIDE's exit code is a bit mask (1 = added, 2 = removed, 4 = changed), so a non-zero code is itself an alert signal.

```
aide --config=/etc/aide/aide-lab.conf --check > /root/aide-tamper-report.txt; echo "exit
code: $?"
grep -E '^[fdl].*: /|^File: ' /root/aide-tamper-report.txt | head -20
grep -E '^(Added|Removed|Changed) entries|Total number' /root/aide-tamper-report.txt
```

Expect `index.html`, `/etc/passwd`, `/etc/shadow` and `/etc/group` as changed and `/usr/local/bin/.sysupdate` as added.

7. Respond: restore and quarantine

Restore the page from the known-good copy, quarantine the backdoor (move it, strip its permissions, keep it as evidence) and lock the rogue account pending investigation.

```
cp -a /root/known-good/index.html /srv/webapp/index.html
mv /usr/local/bin/.sysupdate /root/quarantine/sysupdate.evidence && chmod 000
/root/quarantine/sysupdate.evidence
sha256sum /root/quarantine/sysupdate.evidence 2>/dev/null || ls -l /root/quarantine/
usermod -L -s /usr/sbin/nologin svc-update
aide --config=/etc/aide/aide-lab.conf --check | grep -E '^(Added|Removed|Changed)
entries|^f'
```

The backdoor no longer appears and the page's SHA-256 matches the baseline again; only its ctime still differs, because restoring a file is itself a change. The account changes remain until the change board decides what to do with them.

8. Re-baseline after the approved change

The change board approves keeping `svc-update` locked for forensics. Verify the stored baseline against its off-host hash, update the database to the new known state and confirm that a check is clean again.

```
sha256sum -c /root/offhost/aide-db.sha256
aide --config=/etc/aide/aide-lab.conf --update >/dev/null; echo "update exit code: $?"
chmod 600 /var/lib/aide/lab/aide.db && cp /var/lib/aide/lab/aide.db.new
/var/lib/aide/lab/aide.db && chmod 400 /var/lib/aide/lab/aide.db
sha256sum /var/lib/aide/lab/aide.db > /root/offhost/aide-db.sha256
aide --config=/etc/aide/aide-lab.conf --check >/dev/null; echo "post-update check exit
code: $?"
```

9. Schedule the check and forward it to syslog

Run the check daily and send the output to syslog with a tag, so that the SIEM (Lab 33) can alert on it. Run it once now and read the result back from the journal; on the re-baselined system AIDE reports no differences.

```
cat > /etc/cron.d/aide-lab <<'EOF'
# Daily FIM check at 03:15, results to syslog for the SIEM
15 3 * * * root /usr/bin/aide --config=/etc/aide/aide-lab.conf --check 2>&1 |
/usr/bin/logger -t aide-lab
EOF
/usr/bin/aide --config=/etc/aide/aide-lab.conf --check 2>&1 | /usr/bin/logger -t aide-lab
journalctl -t aide-lab --no-pager -n 20 | grep -iE 'differences|Changed entries' | head -3
```

```
... aide-lab[1234]: AIDE found NO differences between database and filesystem. Looks
okay!!
```

Test it

```
aide --config=/etc/aide/aide-lab.conf --check >/dev/null; echo "exit=$?"
echo "drift" >> /srv/webapp/app.ini
aide --config=/etc/aide/aide-lab.conf --check | grep -c 'app.ini'
sed -i '$d' /srv/webapp/app.ini
```

Expected: `exit=0` on the clean system, then at least `1` match for `app.ini` after the drift. The last line reverts the content, but the file's inode and timestamps have still changed, so re-run step 8 if you want a clean baseline again.

Troubleshooting

Symptom	Fix
`Configuration error: unknown expression`	Old AIDE versions reject some options. Remove the `gzip_dbout` line and run `--config-check` again.
`database_in ... does not exist`	You skipped the `cp` in step 4. Copy `aide.db.new` to `aide.db`.
The check reports `/etc/shadow` changed on every run	Something updated passwords or expiry (for example `chage`). Confirm that the change was approved, then re-baseline as in step 8.

Challenge

Store the baseline database on a separate read-only location (for example a second loop device mounted `ro`), point `database_in` at it, and show that an attacker who rewrites `/var/lib/aide/lab/aide.db` no longer hides their change.

Reflection

FIM told you that `/usr/local/bin` changed, but not who changed it or how they got in. Which other 4.4 and 4.5 tools (SIEM correlation, EDR, auditd) would you correlate with the AIDE alert, and why is FIM detective rather than preventive?

Exam tips

- FIM detects unauthorised changes by comparing hashes against a known-good baseline. It is detective, not preventive.
- Always baseline a clean build, protect the database (read-only or off-host), and re-baseline after approved changes.
- Tripwire, AIDE, Wazuh syscheck and OSSEC are all FIM tools. The exam may name any of them.
- Integrity = hashing. When a question mentions detecting modified system files, the answer is FIM.

References

- AIDE manual — https://aide.github.io

- NIST SP 800-53 Rev. 5, control SI-7 Software, Firmware, and Information Integrity — https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final
- PCI DSS v4.0 Requirement 11.5.2 (change detection) — https://www.pcisecuritystandards.org
- Wazuh file integrity monitoring — https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/index.html

---